

Arquitectura Software

Laboratorio 02

Augusto Pedicino Florez

Erick Salazar Suarez

Felipe Garrido Flores

Pontificia Universidad Javeriana

2026-05-24

Índice

1. Marco conceptual	4
1.1. Tecnologías	4
1.1.1. Java	4
1.1.2. Spring Boot	4
1.1.3. Maven	4
1.1.4. MariaDB	5
1.1.5. MongoDB	5
1.2. Patrón Arquitectónico	5
1.2.1. Arquitectura Hexagonal	5
1.2.1.1. Estructura y responsabilidades	6
1.2.1.2. Comunicación entre capas	6
2. Diseño	7
2.1. Arquitectura	7
2.1.1. Dualidad REST y CLI	8
2.1.2. Dualidad de persistencia	8
2.2. Diagramas	9
2.2.1. HLD	9
2.2.2. Diagrama Entidad Relación	9
2.2.3. C4 model	9
2.2.3.1. Diagrama de contexto	9
2.2.3.2. Diagrama de contenedores	9
2.2.3.3. Diagrama de componentes	9
2.2.3.4. Diagrama de código	9
2.2.3.5. Diagrama de despliegue	9
2.2.3.6. Diagrama de secuencia/dinámico	9
2.2.3.7. Diagrama de System landscape	9
2.3. Decisiones de diseño	9
2.3.1. input-adapter REST y CLI	9
2.3.2. Contenerización con Docker	9
3. Procedimiento	9
3.1. Configuración de la base de datos y creación de entidades y relaciones	9
3.1.1. Implementación y estructura Hexagonal API REST y CLI	9
3.2. Tag y Release	9
3.3. Despliegue y construcción del proyecto	9
3.3.1. Links del despliegue	9
3.3.2. Pasos para correr el proyecto	9
3.4. Producto final	10
3.4.1. API REST	10
3.4.2. CLI (Command Line Interface)	10

4. Conclusiones y lecciones aprendidas	10
4.1. Conclusiones	10
4.2. Lecciones aprendidas	10
5. Referencias	10

Marco conceptual

1.1. Tecnologías

1.1.1. Java

Java es un lenguaje de programación orientado a objetos, de propósito general y fuertemente tipado, desarrollado originalmente por Sun Microsystems y actualmente mantenido por Oracle. Una de sus características más destacadas es su portabilidad: el código Java se compila a *bytecode*, que es ejecutado por la Máquina Virtual de Java (JVM), permitiendo que una misma aplicación corra en cualquier sistema operativo sin modificaciones.

En el contexto del laboratorio, Java actúa como el lenguaje base de toda la aplicación. Su sistema de tipos estricto, su soporte para interfaces y su ecosistema de herramientas lo hacen especialmente adecuado para implementar los principios de la arquitectura hexagonal, donde la separación entre contratos (interfaces) e implementaciones es fundamental.

1.1.2. Spring Boot

Spring Boot es un framework de desarrollo de aplicaciones Java construido sobre el ecosistema Spring. Su principal objetivo es simplificar la configuración y el arranque de aplicaciones, eliminando la necesidad de configuración XML extensa mediante convenciones y autoconfiguración. Permite crear aplicaciones listas para producción con un servidor embebido (Tomcat o Netty) y una estructura de proyecto estandarizada.

Para este laboratorio, Spring Boot provee el contenedor de inyección de dependencias (IoC), la integración con JPA para MariaDB, la integración con Spring Data MongoDB, la creación y definición de una CLI (Command Line Interface) o interfaz por línea de comandos, todo esto utilizando anotaciones como `@SpringBootApplication` y realizando la implementación de la interfaz `CommandLineRunner`. También tiene soporte para exponer servicios REST mediante anotaciones como `@RestController`. La autoconfiguración de Spring Boot permite conectar los adaptadores de entrada y salida con el núcleo de dominio de forma declarativa, sin acoplar las capas entre sí.

1.1.3. Maven

Maven es una herramienta de gestión y construcción de proyectos Java. Permite declarar las dependencias de un proyecto en un archivo `pom.xml` (Project Object Model), gestionar su descarga desde repositorios centrales, y definir el ciclo de vida del build (compilación, pruebas, empaquetado e instalación). Además, soporta proyectos multi-módulo, donde un POM padre agrega varios módulos hijos con dependencias entre ellos.

En el laboratorio, Maven organiza los ocho módulos del proyecto en un reactor multi-módulo. El POM raíz define las propiedades y dependencias compartidas, mientras

que cada módulo (dominio, aplicación, adaptadores de entrada y salida) tiene su propio `pom.xml` que hereda del padre. Esto garantiza que las versiones de las dependencias sean consistentes en todo el proyecto y que el orden de compilación sea el correcto.

1.1.4. MariaDB

MariaDB es un sistema de gestión de bases de datos relacional (RDBMS) de código abierto, derivado de MySQL. Organiza la información en tablas con filas y columnas, y permite definir relaciones entre entidades mediante claves primarias y foráneas. Utiliza SQL estándar para las operaciones de definición (DDL) y manipulación de datos (DML).

En el laboratorio, MariaDB almacena las entidades del dominio (persona, profesión, teléfono y estudios) en tablas relacionales con integridad referencial. La integración con Spring Boot se realiza mediante Spring Data JPA e Hibernate como proveedor ORM, que mapea las entidades Java a tablas de la base de datos sin necesidad de escribir SQL manualmente para las operaciones básicas.

1.1.5. MongoDB

MongoDB es un sistema de gestión de bases de datos NoSQL orientado a documentos. En lugar de tablas y filas, almacena la información en documentos con formato BSON (Binary JSON), agrupados en colecciones. Esta estructura flexible permite almacenar datos sin un esquema fijo, lo que facilita la evolución del modelo de datos sin migraciones complejas.

En el laboratorio, MongoDB almacena las mismas entidades del dominio que MariaDB, pero representadas como documentos en colecciones. La integración con Spring Boot se realiza mediante Spring Data MongoDB, que permite definir repositorios con métodos de consulta declarativos sobre las colecciones. La coexistencia de ambas bases de datos en la misma aplicación ilustra cómo la arquitectura hexagonal desacopla el dominio de la tecnología de persistencia utilizada.

1.2. Patrón Arquitectónico

1.2.1. Arquitectura Hexagonal

La arquitectura hexagonal, también conocida como *Ports and Adapters* y propuesta por Alistair Cockburn, es un estilo arquitectónico cuyo objetivo principal es aislar la lógica de negocio del dominio de cualquier tecnología externa, ya sea una base de datos, una interfaz de usuario, un servicio REST o una línea de comandos. La idea central es que el dominio no debe conocer ni depender de los detalles de infraestructura: son los adaptadores externos los que se adaptan al dominio, y no al revés.

El nombre «hexagonal» no tiene un significado matemático específico: el hexágono representa visualmente el núcleo de la aplicación con múltiples puntos de conexión hacia el exterior. Cada lado del hexágono puede conectarse a un adaptador diferente, lo que hace explícito que la aplicación puede ser conducida por múltiples tecnologías de entrada y salida de forma simultánea e intercambiable.

1.2.1.1. Estructura y responsabilidades

La arquitectura se organiza en las siguientes capas o módulos, cada uno con una responsabilidad bien definida:

- **Common:** Contiene elementos transversales compartidos por todos los módulos, como anotaciones personalizadas (@UseCase, @Adapter, @Port, @Mapper), excepciones comunes y enumeraciones de configuración. No depende de ningún otro módulo del proyecto.
- **Domain:** Es el núcleo de la aplicación. Contiene las entidades del negocio (como Person, Phone, Study, Profession). También define las interfaces de los puertos de entrada (PersonInputPort) y de salida (PersonOutputPort), que representan los contratos que el dominio expone hacia el exterior.
- **Application:** Contiene los casos de uso, que son las implementaciones de los puertos de entrada. Clases como PersonUseCase orquestan la lógica de negocio invocando los puertos de salida sin conocer qué tecnología concreta los implementa. Este módulo depende únicamente de domain y common.
- **Output Adapters (maria-output-adapter, mongo-output-adapter):** Implementan los puertos de salida definidos en el dominio. Cada adaptador traduce entre el modelo de dominio y el modelo de persistencia específico de su tecnología (entidades JPA para MariaDB, documentos para MongoDB), utilizando clases *mapper* dedicadas.
- **Input Adapters (rest-input-adapter, cli-input-adapter):** Implementan los puertos de entrada desde el exterior. El adaptador REST expone endpoints HTTP mediante controladores Spring, mientras que el adaptador CLI presenta un menú interactivo en consola. Ambos traducen las solicitudes externas en llamadas a los casos de uso del módulo application.

1.2.1.2. Comunicación entre capas

El flujo de una solicitud sigue siempre la misma dirección: desde el adaptador de entrada hacia el dominio, y desde el dominio hacia el adaptador de salida, nunca al revés.

Cuando un usuario realiza una petición (ya sea por REST o por el menú CLI), el adaptador de entrada correspondiente la recibe y la transforma al modelo del dominio. Luego invoca el caso de uso apropiado del módulo application, eligiendo previamente qué adaptador de salida utilizar (MariaDB o MongoDB) mediante el método `setPersistencia()`. El caso de uso ejecuta la lógica de negocio y delega la operación de persistencia al puerto de salida. El adaptador de salida seleccionado traduce la entidad de dominio a su representación específica (entidad JPA o documento MongoDB) y ejecuta la operación en la base de datos. La respuesta recorre el camino inverso hasta llegar al usuario.

Esta comunicación unidireccional y mediada por interfaces garantiza que ninguna capa conozca los detalles de implementación de las demás, cumpliendo el principio de inversión de dependencias.

Diseño

2.1. Arquitectura

La aplicación sigue una arquitectura hexagonal (*Ports and Adapters*), organizada como un proyecto multi-módulo de Maven. El núcleo de la aplicación es el dominio, que contiene las entidades de negocio y los contratos (puertos) que definen cómo se comunica con el exterior. Alrededor del dominio se ubican los adaptadores de entrada, que reciben solicitudes desde el mundo exterior, y los adaptadores de salida, que se comunican con los sistemas de persistencia. Ninguna capa interna conoce los detalles de implementación de las capas externas: toda comunicación se realiza a través de interfaces.

La aplicación puede ser conducida por dos adaptadores de entrada de forma independiente: una API REST (disponible en el puerto 3000) y una interfaz de línea de comandos (CLI) con menú interactivo. Ambos adaptadores comparten el mismo núcleo de dominio y los mismos casos de uso, y pueden enrutar sus operaciones hacia cualquiera de los dos motores de persistencia disponibles (MariaDB o MongoDB) en tiempo de ejecución, sin modificar el código del dominio.

Módulo	Tecnología	Responsabilidad
domain	Java puro	Entidades de negocio y contratos (puertos de entrada y salida)
application	Spring (IoC)	Casos de uso: orquestación de la lógica de negocio
common	Java puro	Anotaciones, excepciones y utilidades compartidas
rest-input-adapter	Spring Web / REST	Endpoints HTTP, serialización JSON
cli-input-adapter	Spring Boot CLI	Menú interactivo por consola
maria-output-adapter	Spring Data JPA / MariaDB	Persistencia relacional
mongo-output-adapter	Spring Data MongoDB	Persistencia documental

Tabla 1: Módulos de la arquitectura hexagonal y tecnologías asociadas

2.1.1. Dualidad REST y CLI

Una decisión central del diseño fue implementar dos adaptadores de entrada independientes que exponen la misma lógica de negocio mediante interfaces distintas:

- **Adaptador REST (`rest-input-adapter`):** Contiene controladores Spring anotados con `@RestController` que reciben peticiones HTTP y retornan respuestas en formato JSON. Es el punto de entrada para clientes externos, aplicaciones web o herramientas de prueba como Swagger UI.
- **Adaptador CLI (`cli-input-adapter`):** Implementa `CommandLineRunner` de Spring Boot y presenta un menú jerárquico interactivo en la consola. Permite operar sobre las mismas entidades (Persona, Profesión, Teléfono, Estudio) seleccionando además el motor de persistencia (MariaDB o MongoDB) en cada sesión.

Ambos adaptadores inyectan los mismos casos de uso del módulo `application` a través de los puertos de entrada definidos en `domain`, garantizando que la lógica de negocio subyacente sea idéntica independientemente del canal de acceso utilizado.

2.1.2. Dualidad de persistencia

Otra decisión relevante fue soportar dos motores de persistencia de forma simultánea dentro de la misma ejecución de la aplicación. El adaptador de salida para MariaDB (`maria-output-adapter`) utiliza Spring Data JPA con Hibernate como ORM, mapeando las entidades del dominio a tablas relacionales. El adaptador de salida para MongoDB (`mongo-output-adapter`) utiliza Spring Data MongoDB, representando las mismas entidades como documentos en colecciones.

La selección del motor de persistencia activo no requiere reiniciar la aplicación ni cambiar configuración: el adaptador de entrada invoca `setPersistencia()` sobre el caso de uso antes de cada operación, inyectando el puerto de salida correspondiente. Esto es posible gracias a que ambos adaptadores implementan la misma interfaz `PersonOutputPort` (y sus equivalentes para las demás entidades), definida en el dominio.

2.2. Diagramas

2.2.1. HLD

2.2.2. Diagrama Entidad Relación

2.2.3. C4 model

2.2.3.1. Diagrama de contexto

2.2.3.2. Diagrama de contenedores

2.2.3.3. Diagrama de componentes

2.2.3.4. Diagrama de código

2.2.3.5. Diagrama de despliegue

2.2.3.6. Diagrama de secuencia/dinámico

2.2.3.7. Diagrama de System landscape

2.3. Decisiones de diseño

2.3.1. input-adapter REST y CLI

<[llenar input-adapter REST y CLI](#)>

2.3.2. Contenerización con Docker

<[llenar contenerizacion con Docker](#)>

Procedimiento

3.1. Configuración de la base de datos y creación de entidades y relaciones

<[llenar configuracion de la DB y entidades](#)>

3.1.1. Implementación y estructura Hexagonal API REST y CLI

<[llenar implementacion](#)>

3.2. Tag y Release

<[llenar tag y release](#)>

3.3. Despliegue y construcción del proyecto

<[llenar despliegue y construccion del proyecto](#)>

3.3.1. Links del despliegue

<[llenar links del despliegue](#)>

3.3.2. Pasos para correr el proyecto

<[llenar pasos para correr el proyecto](#)>

- El **Dockerfile** especifica como se construye la **imagen** del **proyecto** utilizando .NET 10.
- El archivo **compose.yml** especifica como **levantar** todos los **servicios** o **contenedores** necesarios para que el proyecto se ejecute de manera correcta.

1. **clonar el repositorio:**

```
git clone https://github.com/ErickSalazar07/arquitectura-lab-01.git
```

2. **construcción y ejecución del proyecto:**

```
docker compose up
```

3.4. Producto final

3.4.1. API REST

<llenar producto final – API REST>

3.4.2. CLI (Command Line Interface)

<llenar producto final CLI>

Conclusiones y lecciones aprendidas

4.1. Conclusiones

<llenar conclusiones>

4.2. Lecciones aprendidas

<llenar lecciones aprendidas>

Referencias

<llenar referencias>